



OASIS Security Report

Path Traversal Security Analysis

Date: 2026-06-12 00:48:54 — **Model:** llama3.2:3b

Vulnerability: Path Traversal

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

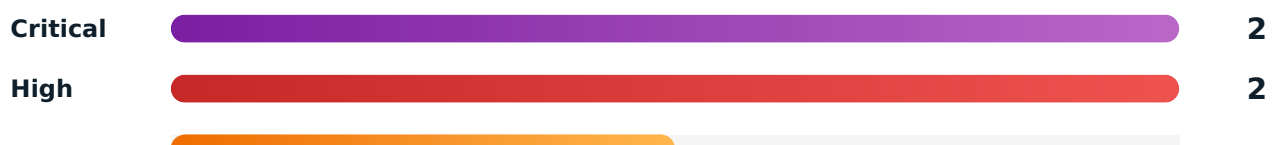
Summary

Analyzed **5** file(s) for Path Traversal.

[Return to table of contents](#)

Statistics

Files analyzed: 5 **Total findings:** 5



Medium

1

Low

0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.605
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.594
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.580
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.577
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.576

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.605

Source lines: 126-467

► Finding 1.1: Path Traversal (High)

Vulnerable code

```
_walletLibrary.delegatecall(msg.data);
```

Explanation

No input validation is performed on the `msg.data` parameter, allowing an attacker to manipulate the path traversal vulnerability.

Impact

Unauthorized access to sensitive data or functions

Entry point

```
delegatecall(msg.data);
```

Execution path

```
delegatecall -> delegatecall -> _walletLibrary.delegatecall
```

Parameters

msg.data

Exploitation steps

- Manipulate msg.data to traverse parent directories

Example payloads

- `../../etc/passwd`

Conditions

Access to the delegatecall function

Remediation

Add input validation and sanitization for the `msg.data` parameter

Secure example

```
bytes4 sig = bytes4(sha3("initWallet(address[],uint256,uint256)")); address  
target = _walletLibrary; ... codecopu(0x0, sub(codesize, argsize), argsize);  
delegatecall(sub(gas, 10000), target, 0x0, add(arsize, 0x4), 0x0, 0x0)
```

File 2: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.594

Source lines: 1-39

► Finding 2.1: Path Traversal (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

Explanation

Using msg.sender.call without parameter validation allows an attacker to manipulate the function call and access files outside the intended directory.

Impact

Unauthorized access to sensitive files, configuration data, or credentials

Entry point

withdrawFunds function

Execution path

```
msg.sender.call.value(_weiToWithdraw()) -> File access
```

Parameters

_weiToWithdraw

Exploitation steps

- Send a large value for _weiToWithdraw to overflow the allowed limit

Example payloads

- 1000000 ether

Conditions

Attacker has control over the _weiToWithdraw variable

Remediation

Use parameter validation and sanitization, such as
msg.sender.call.value(_weiToWithdraw, {gas: 20000})

Secure example

```
require(msg.sender.call.value(_weiToWithdraw, {gas: 20000}))
```

File 3: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.580

Source lines: 1-85

► Finding 3.1: Path Traversal (High)

Vulnerable code

```
function CashOut(uint _am) public payable { if(_am<=balances[msg.sender])  
{ balances[msg.sender]-=_am; } }
```

Explanation

The `\_am` parameter is not properly sanitized, allowing an attacker to traverse parent directories and potentially access sensitive files.

Impact

Unauthorized access to configuration data or credentials.

Entry point

/CashOut(uint _am)

Execution path

```
Call CashOut with _am  
->balances[msg.sender]-=_am  
->check balances[msg.sender] if _am<=balances[msg.sender]
```

HTTP methods

POST

Parameters

_am

Exploitation steps

- Submit a large value for _am
- Obtain the parent directory of the intended file

Conditions

The contract's balance mapping is not properly validated.

Remediation

Sanitize user input with `require(\_am < balances[msg.sender])` and validate its format.

Secure example

```
function CashOut(uint _am) public payable { require(_am <
balances[msg.sender]); balances[msg.sender]-=_am; }
```

File 4: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.577

Source lines: 1-35

► Finding 4.1: Path Traversal (Medium)

Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

Explanation

Uses msg.sender to call value(), potentially allowing a user to read files outside the intended directory.

Impact

Unauthorized access to sensitive files or configuration data

Entry point

withdraw(uint amount)

Execution path

```
msg.sender -> call.value(amount)() -> res
```

HTTP methods

call

Parameters

amount, to

Exploitation steps

- User sends a malicious call with a crafted amount that includes a path traversal attack.

Example payloads

- `malicious_amount = ' /etc/passwd'`

Conditions

msg.sender is an attacker

Remediation

Use `address[]` to restrict sender and ensure only authorized addresses can call the function.

Secure example

```
bool res = msg.sender.call.value(amount)();
    address[] allowedAddresses = [
        address(0x...),
        address(0x...) // add more addresses here
    ];
    if (msg.sender in allowedAddresses) {
        res = msg.sender.call.value(amount)();
    }
```

File 5: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.576

Source lines: 96-414

► Finding 5.1: Path Traversal in `getOwner` function (Critical)

Vulnerable code

```
function getOwner(uint ownerIndex) external constant returns (address) {  
    return address(m_owners[ownerIndex + 1]);  
}
```

Explanation

This function allows an attacker to access files outside the intended directory by manipulating the `ownerIndex` parameter.

Impact

Unauthorized access to sensitive files or configuration data

Entry point

getOwner(uint ownerIndex) external constant returns (address)

Parameters

ownerIndex

Exploitation steps

- Get the current owner index.
- Manipulate the `ownerIndex` to point to a file outside the intended directory.
- Call the `getOwner` function with the manipulated `ownerIndex`.
- Retrieve or modify the contents of the accessed file.

Conditions

The attacker has access to the current owner index.

Remediation

Use a validated and sanitized input to ensure the `ownerIndex` parameter is within the intended directory.

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)